



Corrigindo um Jogo da Velha em Python

Guia Visual Passo a Passo com Tkinter **Simples**



1 Introdução

Entendendo o Projeto

Recriaremos, criamos um jogo da velha com interface gráfica usando:

- Python
- Tkinter
- Classes (objetos)
- Funções (def)
- Botões
- Canvas (QT)
- Listas
- Mensagem na tela



❗ O código possui alguns erros que impedem o jogo de funcionar corretamente. Agora vamos corrigir cada problema passo a passo.

2 O Problema

Código com Erros

- ❗ `__init__` criado incorretamente
- ❗ `geometry()` sem `self.root`
- ❗ Apas não fechadas
- ❗ Passar sem `self.root`
- ❗ Tabuleiro criado apenas 3 botões
- ❗ Configurar inicialmente no computador
- ❗ Verificação de empate errada
- ❗ Retorno do jogo incorreto
- ❗ Mensagem de vitória errada

❗ Basta esses poucos erros e o jogo não funciona como deveria. Vamos corrigir um por um!

3 Passo 1 — Corrigir o `__init__`

Como estava (errado):

```
def __init__(self, root):
```

Como corrigir:

```
def __init__(self, root):
```

1 Explicação

- ❗ `self` representa o próprio objeto da classe.
- ❗ `root`:
- o Python não entende o objeto `root` sozinho;
- os atributos não funcionam;
- o programa gera uma instância vazia.

4 Passo 2 — Corrigir o `geometry()`

Como estava (errado):

```
geometry("300x300")
```

Como corrigir:

```
self.root.geometry("300x300")
```

Problemas encontrados

- ❗ `self.root` não existe
- ❗ apas não foram fechadas
- ❗ `self.root`

2 O que aprendemos

❗ `geometry()` define o tamanho do objeto de Tkinter.
Formato: "LARGURA x ALTURA"

5 Passo 3 — Corrigir o `Player (tela)`

Como estava (errado):

```
self.root. (self.vitorias_player) +  
    PC (self.vitorias_pc)
```

Como corrigir:

```
self.root. (self.vitorias_player) +  
    PC (self.vitorias_pc)
```

1 Explicação

- ❗ `self` não pode ser usado para atribuir valores diretamente.
- ❗ `self` e `self` não podem ser usados.
- ❗ `self.vitorias_player`

6 Passo 4 — Corrigir as Botões do Tabuleiro

Como estava (errado):

```
for i in range(9):
```

Como corrigir:

```
for i in range(9):
```

1 Explicação

- ❗ `for` deve ser usado.
 - 3 linhas
 - 3 colunas
- Nota: `0 < i < 9 = 9` pessoas
Com `range(9)`, o número de botões do tabuleiro é corrigido.

7 Passo 5 — Corrigir a Jogada do Computador

Como estava (errado):

```
if not vitorias:
```

Como corrigir:

```
if None:
```

1 Explicação

- ❗ `if` deve garantir se existe vitória.
- ❗ `computador` só pode jogar se não houver jogadores disponíveis.

8 Passo 6 — Corrigir o `Empate`

Como estava (errado):

```
if ** == self.tabuleiro:
```

Como corrigir:

```
if ** == self.tabuleiro:
```

6 Explicação

- ❗ `Empate` acontece quando:
- não existem jogadores
- não existem jogadores disponíveis

9 Passo 7 — Corrigir o `Retorno`

Como estava (errado):

```
self.jogo.ativar = False
```

Como corrigir:

```
self.jogo.ativar = True
```

1 Explicação

- ❗ Quando `retornar`:
- o jogo precisa voltar o funcionamento.
- por isso o valor deve ser `True`.

10 Passo 8 — Corrigir a Mensagem Final

Como estava (errado):

```
"Parabéns! Você ganhou!"
```

Como corrigir:

```
"Parabéns! Você venceu!"
```

1 Explicação

- ❗ A mensagem estava incorreta.
- ❗ Quando o jogador "PC" vence, a mensagem correta deve informar vitória.

11 Novidades Aprendidas Nesse Projeto



Class (objetos) e como criar uma instância de uma classe.



Tkinter: interface gráfica com botões e botões.



Botões (Tkinter): como criar uma instância de um botão.



`self` (Tkinter): como criar uma instância de um objeto.



`self` (Tkinter): como criar uma instância de um objeto.



`self` (Tkinter): como criar uma instância de um objeto.

12 Conclusão

O Que Aprendemos

- Como criar um Python
- Usar Tkinter
- Trabalhar com botões
- Criar funções
- Criar variáveis (if)
- Como lidar com gráficos
- Manipular listas
- Analisar o jogo
- Retornar o jogo corretamente

Resultado Final

- o tabuleiro funciona corretamente
- o computador joga corretamente
- o jogador humano
- o jogo retorna o vencedor